



Module Title

Cybersecurity and Networking

Internship Assignment Submission

Threat Modeling & Secure Design

Student Name: Jiwosh Shrestha

Internship ID: JVL-CS-2026-8990

Role: Cybersecurity Intern

Duration: 3 months

Assignment Submission Date: Wednesday, 7th October 2026

Day:31 (Thirty-One)

Table of Contents

1.DAY 27 ASSIGNMENT A05:2025 INJECTION — SECURE INPUT HANDLING	3
1.1 PART A — FUNDAMENTALS	3
1.2 PART B — DEFENSIVE CONTROLS	5
1.3 PART C — SAFE PRACTICAL ANALYSIS	7
1.4 PART D — DESIGN COMPARISON	8

1.DAY 27 ASSIGNMENT A05:2025 INJECTION — SECURE INPUT HANDLING

1.1 PART A — FUNDAMENTALS

1.What is Injection?

Injection happens when untrusted user input is treated as a part of a command query, or instruction. This can change the intended behavior of an application.

2. What is untrusted input?

Untrusted input is data received from users or external sources that should not be automatically trusted. It should be properly validated and handles before use.

3. Give five examples of user-controlled input.

Login fields

Search boxes

URL parameters

API request data

Cookies

4. What is an interpreter?

An interpreter is a system that processes instructions written in a specific language or syntax. Examples include a database engine for SQL and an operating system shell for commands.

5. Explain the difference between Data and Instructions.

Data is information that should be treated as a value, while instructions tell a system what action to perform. Secure applications should keep user data separate from commands or instructions.

6. What is SQL Injection at a conceptual level?

SQL Injection occurs when untrusted input is directly included in an operating system command. The input may then influence how the command is interpreted or executed.

7. What is OS Command Injection at a conceptual level?

OS command Injection occurs when untrusted input is directly included in an operating-system command. The input may then influence how the command is interpreted or executed.

8. Why is raw string concatenation with untrusted input risky?

It mixes user-controlled data with commands or queries. This can change the intended meaning of the instruction and create an injection risk.

1.2 PART B — DEFENSIVE CONTROLS

9. What is a parameterized query?

A parameterized query keeps the SQL query structure separate from the user-provided data. The user value is passed as a parameter instead of being directly added to the query.

10. What is a prepared statement?

A prepared statement is a method where the query structure is prepared first, then the required data is bound to it before execution.

11. Why do parameterized queries improve security?

They separate SQL instructions from user data. This helps prevent user input from being interpreted as SQL commands.

12. What is input validation?

Input validation checks whether user input follows the expected type, format, length, range and allowed values before it is processed.

13. What is allowlist validation?

Allowlist validation defines when input is allowed and accepts only those expected values. For example, an application may allow only the roles student, reviewer, and admin.

14. Why is client-side validation alone not sufficient?

Client server validation happens in the user's browser and can be bypassed or changed. Therefore, it should not be treated as a security boundary.

15. Why should important validation also happen server-side?

The server is responsible for securely processing the request, so it must independently check important input. This prevents invalid or manipulated data from being trusted.

16. Explain Least Privilege for a database account.

Least Privilege means giving the database account only the permissions required for its actual tasks. It should not have unnecessary administrator-level access.

17. Why should detailed database errors not normally be displayed publicly?

Detailed errors can reveal sensitive database or server information. It is safer to show a simple error message and keep details in secure logs.

1.3 PART C — SAFE PRACTICAL ANALYSIS

Fictional application: Student Search Portal Input: Student ID Current Design: - Browser performs basic validation - Backend trusts the value - Raw input is concatenated into a database query - Database account has administrator privileges - Detailed database errors are shown publicly

1. Identify every security weakness:

1) Raw input concatenated directly into database query string; 2) Backend trusts client-side validation blindly; 3) Database user possesses unnecessary administrator privileges; 4) Detailed database error messages rendered publicly.

2. Which OWASP Top 10:2025 category primarily applies? A05:2025 — Injection.

3. Why is client-side validation insufficient? Client-side controls run in the untrusted browser environment. Adversaries can bypass browser validation using intercepting proxies (e.g., Burp Suite) or cURL to send malicious raw inputs directly to the server.

4. What should replace raw query concatenation? Parameterized queries (prepared statements) that strictly isolate SQL code logic from dynamic user parameters.

5. What server-side validation could be applied to Student ID? Strict allowlist regex validation, combined with backend integer type conversion before query binding.

6. How should database privileges be improved? Enforce Least Privilege by restricting database user accounts to read-only SELECT access specifically on required tables, revoking administrator permissions.

7. How should public error handling be improved? Replace technical error messages with generic user-friendly notices while routing detailed technical logs to secure server storage.

1.4 PART D — DESIGN COMPARISON

Design A: Query structure is prepared separately. Student ID is passed as a bound parameter. Design B: Raw Student ID is concatenated directly into query text.

Answer:

1. Which design is safer?

Design A is safer.

2. Explain why.

Design A keeps the query structure separate from the Student ID by using a bound parameter. This prevents the Student ID from being treated as part of the SQL query structure.

3. Which security principle does the safer design follow?

It follows the principle of separating data from commands or queries or queries. It also follows the secure practice of using parameterized queries.